

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, LabelEncoder

import seaborn as sns

import matplotlib.pyplot as plt


# Load dataset

file_path =

r"C:\\Users\\brian\\OneDrive\\Escritorio\\Lambton\\Semester2\\Toxicology\\Datasets\\PubChem_com
pound_cache.csv"

data = pd.read_csv(file_path)


# Step 1: Handle Missing Values

data.fillna(data.median(numeric_only=True), inplace=True) # Fill numeric columns with median
data.fillna("Unknown", inplace=True) # Fill non-numeric columns with 'Unknown'


# Step 2: Drop Irrelevant Columns

columns_to_drop = ['cmpdsynonym', 'iupacname', 'meshheadings', 'annotation']

data.drop(columns=columns_to_drop, inplace=True, errors='ignore')


# Step 3: Label Encode Categorical Variables

categorical_columns = data.select_dtypes(include=['object']).columns

for col in categorical_columns:

    data[col] = LabelEncoder().fit_transform(data[col])


# Step 4: Select Features and Target

# Replace 'annotation' with your actual target column if applicable
```

```
target_column = 'mw' # Example: use molecular weight for testing (update with your target variable)

if target_column not in data.columns:

    raise ValueError("Specify a valid target column.")

X = data.drop(columns=[target_column])

y = data[target_column]
```

```
#####3
```

```
# Preparing features and target for the first model (e.g., is_toxic classification)

data['is_toxic'] = (data['hbondddonor'] > 0).astype(int) # Example logic for toxicity
```

```
# Define features and target for Model 2
```

```
features_model_2 = ['mw', 'polararea', 'complexity', 'xlogp', 'heavycnt',

                    'hbondddonor', 'rotbonds', 'totalatomstereocnt', 'totalbondstereocnt']

target_model_2 = 'hbondddonor' # Update this if your target column name is different
```

```
# Prepare data for Model 2
```

```
X2 = data[features_model_2]

y2 = data[target_model_2]
```

```
# Train/Test Split
```

```
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.2, random_state=42)
```

```
# Train RandomForestRegressor
```

```
from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_absolute_error, r2_score
```

```
model_2 = RandomForestRegressor(random_state=42)

model_2.fit(X2_train, y2_train)


print(data[features_model_2].isnull().sum())

print(data[target_model_2].isnull().sum())


X2 = data[features_model_2].fillna(data[features_model_2].mean())

y2 = data[target_model_2].fillna(data[target_model_2].mean())


print(f"Length of X2: {len(X2)}, Length of y2: {len(y2)}")


X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.2, random_state=42)

model_2 = RandomForestRegressor(random_state=42)

model_2.fit(X2_train, y2_train)


y2_pred = model_2.predict(X2_test)

print(f"Model 2 MAE: {mean_absolute_error(y2_test, y2_pred)}")

print(f"Model 2 R2 Score: {r2_score(y2_test, y2_pred)}")


# Save Model 2

import os

import joblib

output_dir = r"C:\\Users\\brian\\OneDrive\\Escritorio\\Lambton\\Semester2\\Toxicology"

joblib.dump(model_2, os.path.join(output_dir, 'model_2_toxicity_level.pkl'))
```